

Übungsblatt 4

HTTP und REST

5430UE Web and Data Engineering

24. März 2026

Prof. Dr. Michael Granitzer

Modul A — HTTP

Übung 4.A.1: HTTP-Methoden und Status-Codes

Ordnen Sie jeder Situation die korrekte HTTP-Methode und den passenden Status-Code zu:

Situation	Methode	Status-Code
Produktliste abrufen	?	?
Neues Produkt anlegen	?	?
Produktname aktualisieren (nur ein Feld)	?	?
Produkt vollständig ersetzen	?	?
Produkt löschen	?	?
Ressource nicht gefunden	—	?
Validierungsfehler im Request-Body	—	?
Server-interner Fehler	—	?

Übung 4.A.1: HTTP-Methoden und Status-Codes — Lösung

Situation	Methode	Status-Code
Produktliste abrufen	GET	200 OK
Neues Produkt anlegen	POST	201 Created
Produktname aktualisieren (Teilupdate)	PATCH	200 OK
Produkt vollständig ersetzen	PUT	200 OK
Produkt löschen	DELETE	204 No Content
Ressource nicht gefunden	—	404 Not Found
Validierungsfehler	—	400 Bad Request
Server-interner Fehler	—	500 Internal Server Error

Merkhilfe: POST = neu erzeugen (URL unbekannt). PUT = ersetzen (URL bekannt). PATCH = partiell ändern.

Übung 4.A.2: Caching-Strategien

Ein Online-Shop liefert verschiedene Ressourcen. Bestimmen Sie für jede die geeignete Caching-Strategie:

Ressource	Strategie (kein Cache / kurz / lang / ewig)	HTTP-Header
Produktbild <code>logo.png</code> (unveränderlich, mit Hash im Dateinamen)	?	?
Produktliste (aktualisiert sich stündlich)	?	?
Warenkorb-Inhalt (nutzerspezifisch)	?	?
JavaScript-Bundle <code>app.abc123.js</code> (Hash im Namen)	?	?

Übung 4.A.2: Caching-Strategien — Lösung

Ressource	Strategie	HTTP-Header
logo.png mit Hash	Ewig	Cache-Control: public, max-age=31536000, immutable
Produktliste	Kurz	Cache-Control: public, max-age=3600
Warenkorb	Kein Cache	Cache-Control: private, no-store
app.abc123.js	Ewig	Cache-Control: public, max-age=31536000, immutable

Prinzip: Dateien mit Content-Hash im Namen können unbegrenzt gecacht werden — Änderung = neuer Hash = neue URL.
Nutzerspezifische Daten nie öffentlich cachen (`private`).

Modul B — REST API Design

Übung 4.B.1: RESTful URL-Design

Ein Fitness-App-Backend soll folgende Operationen unterstützen. Entwerfen Sie RESTful URLs:

1. Alle Trainingspläne eines Nutzers abrufen
2. Einen neuen Trainingsplan für einen Nutzer erstellen
3. Eine spezifische Übung aus einem Trainingsplan abrufen
4. Eine Übung in einem Trainingsplan aktualisieren
5. Alle abgeschlossenen Trainingseinheiten eines Nutzers in einem Zeitraum filtern

Welche URL-Muster sind dabei problematisch: `/getTrainings`, `/user/deleteById?id=5?`

Übung 4.B.1: RESTful URL-Design — Lösung

Operation	Methode	URL
Trainingspläne eines Nutzers	GET	/users/{userId}/plans
Neuen Plan erstellen	POST	/users/{userId}/plans
Spezifische Übung abrufen	GET	/users/{userId}/plans/{planId}/exercises/{exerciseId}
Übung aktualisieren	PUT/PATCH	/users/{userId}/plans/{planId}/exercises/{exerciseId}
Einheiten filtern	GET	/users/{userId}/sessions?from=2026-01-01&to=2026-03-31&completed=true

Problematische Muster:

- /getTrainings — Verb in der URL verletzt REST: HTTP-Methoden (GET, POST...) sind die Verben, URLs sind Substantive.
- /user/deleteById?id=5 — Operation im Pfad und Query-Parameter für ID; korrekt: DELETE /users/5.

Übung 4.B.2: HATEOAS und API-Versionierung

1. Was bedeutet **HATEOAS** und welchen praktischen Vorteil bietet es gegenüber einer reinen Daten-API?
Zeigen Sie ein JSON-Beispiel für eine Produktressource mit HATEOAS-Links.
2. Nennen Sie zwei Strategien für **API-Versionierung** und diskutieren Sie je einen Vorteil und Nachteil.

Übung 4.B.2: HATEOAS und API-Versionierung — Lösung

1. HATEOAS (Hypermedia as the Engine of Application State): Die API-Antwort enthält Links zu möglichen Folgeaktionen. Clients müssen URLs nicht hartcodieren.

```
{
  "id": 42,
  "name": "Laptop Pro",
  "price": 999.00,
  "_links": {
    "self": { "href": "/products/42" },
    "reviews": { "href": "/products/42/reviews" },
    "order": { "href": "/orders", "method": "POST" }
  }
}
```

Vorteil: API-URLs können sich ändern, ohne Clients zu brechen — sie folgen Links statt URLs zu kennen.

2. Versionierungsstrategien:

Strategie	Vorteil	Nachteil
URL-Versionierung /api/v1/products	Explizit, einfach zu verstehen und zu testen	Dupliziert Routen; Clients müssen URL-Änderung mitmachen
Header-Versionierung Accept: application/vnd.api+json;version=2	URL bleibt stabil	Schwerer zu testen (kein Browser-direktaufruf), komplexeres Routing

Modul C — REST vs. SOAP

Übung 4.C.1: REST vs. SOAP Entscheidung

Entscheiden Sie für jedes Szenario, ob REST oder SOAP besser geeignet ist, und begründen Sie:

Szenario	REST oder SOAP?	Begründung
Mobile App ruft Produktdaten ab	?	?
Banktransfer zwischen zwei Finanzsystemen (ACID, Transaktionen)	?	?
IoT-Sensoren senden Messwerte (geringe Bandbreite)	?	?
Legacy-Enterprise-Integration mit strenger Vertragsbindung	?	?

Übung 4.C.1: REST vs. SOAP Entscheidung — Lösung

Szenario	Empfehlung	Begründung
Mobile App	REST	Leichtgewichtig, JSON, HTTP-Caching, geringe Latenz
Banktransfer	SOAP	WS-AtomicTransaction, ACID-Garantien, WS-Security für Compliance
IoT-Sensoren	REST	Minimaler Overhead, JSON oder binäre Formate; SOAP-XML zu schwer
Legacy Enterprise	SOAP	WSDL erzwingt stabilen Vertrag; bestehende SOAP-Infrastruktur verwenden

Faustregel: REST für neue, performance-kritische Systeme. SOAP wenn Transaktionssicherheit, formale Verträge oder Legacy-Integration gefordert sind.

Modul D — Sicherheit und API-Dokumentation

Übung 4.D.1: TLS/SSL — Transport-Sicherheit

1. Was war das zentrale Sicherheitsproblem von **SSL 2.0**, das zur Entwicklung von TLS führte?
2. Ist TLS heute für Web-Anwendungen verpflichtend oder optional? Begründen Sie aus technischer und regulatorischer Sicht.
3. Erklären Sie den **TLS-Handshake** in 4 Schritten: Was wird ausgetauscht und welches Ziel hat jeder Schritt?
4. Was bedeutet **Certificate Pinning** und für welche Art von Anwendung ist es besonders relevant?

Übung 4.D.1: TLS/SSL — Transport-Sicherheit — Lösung

1. **SSL 2.0 Schwächen:** Mehrere kritische Sicherheitslücken: schwache MAC-Konstruktion (MD5), anfällig für Downgrade-Angriffe, fehlender Schutz des Handshakes vor Manipulation, unzureichende Trennung von Authentifizierung und Verschlüsselung.
2. **TLS heute:**
 - **Technisch:** HTTP/2 erfordert TLS; moderne Browser markieren HTTP-Seiten als „Nicht sicher“.
 - **Regulatorisch:** DSGVO/GDPR verlangt angemessene technische Schutzmaßnahmen für personenbezogene Daten; PCI-DSS verlangt TLS für Zahlungsdaten.
3. **TLS-Handshake (vereinfacht):**
 1. **ClientHello:** Client sendet unterstützte TLS-Versionen und Cipher Suites
 2. **ServerHello + Certificate:** Server wählt Cipher Suite, sendet X.509-Zertifikat
 3. **Key Exchange:** Beide Seiten einigen sich auf einen gemeinsamen Session-Key
 4. **Finished:** Beide bestätigen, dass der Handshake vollständig und unverändert war
4. **Certificate Pinning:** Die Anwendung akzeptiert nur ein bestimmtes Zertifikat oder dessen Public Key. Besonders relevant für **Mobile Apps**, die Man-in-the-Middle-Angriffe über manipulierte CAs verhindern müssen.

Übung 4.D.2: Swagger/OpenAPI — API-Dokumentation

1. Was ist **OpenAPI** (früher: Swagger) und welches Problem löst es bei der Entwicklung von REST-APIs?
2. Was bedeutet **Design-First** vs. **Code-First** bei REST-APIs? Nennen Sie je einen Vorteil.
3. Gegeben: eine OpenAPI-Spezifikation beschreibt folgenden Endpunkt:

```
/products/{id}:  
get:  
  summary: Get a product by ID  
  parameters:  
    - name: id  
      in: path  
      required: true  
      schema:  
        type: integer  
  responses:  
    '200':  
      description: Product found  
    '404':  
      description: Product not found
```

- a) Welchen HTTP-Status liefert der Endpunkt bei einer nicht vorhandenen Produkt-ID? b) Warum ist `id` als `in: path` (nicht `in: query`) definiert?

Übung 4.D.2: Swagger/OpenAPI — API-Dokumentation — Lösung

1. **OpenAPI/Swagger:** Eine maschinenlesbare Spezifikation (YAML/JSON), die REST-APIs formal beschreibt: Endpunkte, Parameter, Request/Response-Schemata, Authentifizierung. Ohne Spezifikation ist eine API nur durch Quellcode oder informelle Dokumentation verstehbar.
2. **Design-First vs. Code-First:**
 - **Design-First:** API-Spezifikation wird vor dem Code geschrieben. Vorteil: Frontend- und Backend-Teams können parallel arbeiten.
 - **Code-First:** Code wird zuerst geschrieben, OpenAPI-Spec aus Annotations generiert. Vorteil: Spec ist immer aktuell.
3. a) **HTTP 404** — „Product not found“ b) **in:** path bedeutet, die ID ist Teil der URL-Struktur (/products/42), nicht als Query-Parameter (/products?id=42). Ein Pfad-Parameter identifiziert eine spezifische Ressource.